

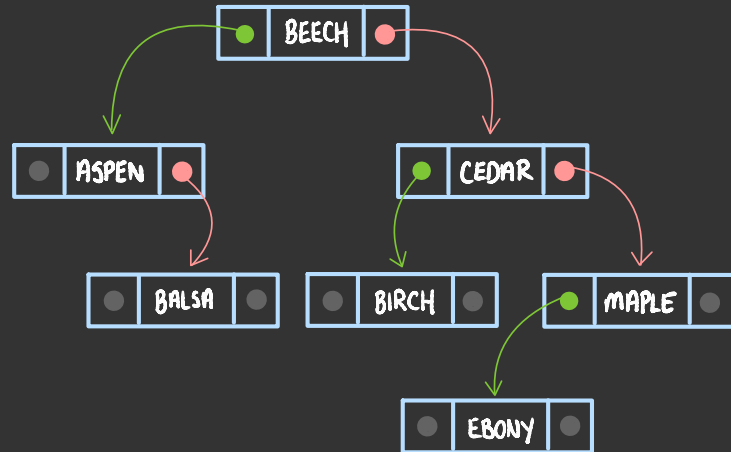
special
methods

CSCI
134

previously on csci 134

we implemented a tree using **lists** of length 3

```
In [1]: balsa = [None, "BALSA", None]
In [2]: aspen = [None, "ASPEN", balsa]
In [3]: birch = [None, "BIRCH", None]
In [4]: ebony = [None, "EBONY", None]
In [5]: maple = [ebony, "MAPLE", None]
In [6]: cedar = [birch, "CEDAR", maple]
In [7]: beech = [aspen, "BEECH", cedar]
```



but instead we could create a new **type**

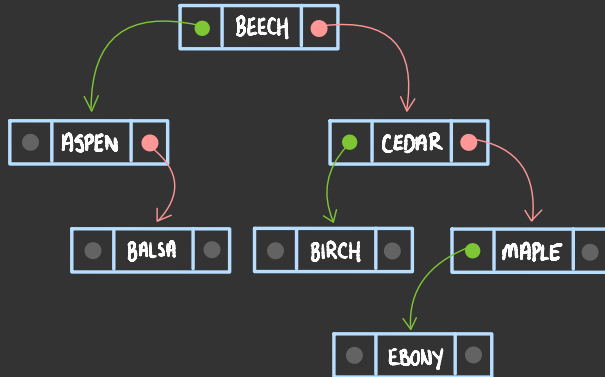
```
def tree_node(l, lbl, r):  
    | return [l, lbl, r]  
  
def leaf(lbl):  
    | return tree_node(None, lbl, None)  
  
def label(node):  
    | return node[1]  
  
def left(node):  
    | return node[0]  
  
def right(node):  
    | return node[2]  
  
def set_label(node, new_label):  
    | node[1] = new_label  
  
def set_left(node, left_node):  
    | node[0] = left_node  
  
def set_right(node, right_node):  
    | node[2] = right_node
```



```
class TreeNode:  
    |  
    | def initialize(self, lbl, l, r):  
    |     self.label = lbl  
    |     self.left = l  
    |     self.right = r
```

but instead we could create a new **type**

```
class TreeNode:
    def initialize(self, lbl, l, r):
        self.label = lbl
        self.left = l
        self.right = r
```



```
balsa = TreeNode()
balsa.initialize("BALSA", None, None)
aspen = TreeNode()
aspen.initialize("ASPEN", None, balsa)
birch = TreeNode()
birch.initialize("BIRCH", None, None)
ebony = TreeNode()
ebony.initialize("EBONY", None, None)
maple = TreeNode()
maple.initialize("MAPLE", ebony, None)
cedar = TreeNode()
cedar.initialize("CEDAR", birch, maple)
beech = TreeNode()
beech.initialize("BEECH", aspen, cedar)
```

wouldn't it be
nice to do **this**
in **one line**?



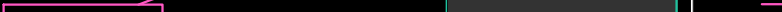
```
balsa = TreeNode()
balsa.initialize("BALSA", None, None)
aspen = TreeNode()
aspen.initialize("ASPEN", None, balsa)
birch = TreeNode()
birch.initialize("BIRCH", None, None)
ebony = TreeNode()
ebony.initialize("EBONY", None, None)
maple = TreeNode()
maple.initialize("MAPLE", ebony, None)
cedar = TreeNode()
cedar.initialize("CEDAR", birch, maple)
beech = TreeNode()
beech.initialize("BEECH", aspen, cedar)
```



```
class TreeNode:

    def initialize(self, lbl, l, r):
        self.label = lbl
        self.left = l
        self.right = r
```

```
balsa = TreeNode()
balsa.initialize("BALSA", None, None)
aspen = TreeNode()
aspen.initialize("ASPEN", None, balsa)
birch = TreeNode()
birch.initialize("BIRCH", None, None)
ebony = TreeNode()
ebony.initialize("EBONY", None, None)
maple = TreeNode()
maple.initialize("MAPLE", ebony, None)
cedar = TreeNode()
cedar.initialize("CEDAR", birch, maple)
beech = TreeNode()
beech.initialize("BEECH", aspen, cedar)
```



```
class TreeNode:

    def __init__(self, lbl, l, r):
        self.label = lbl
        self.left = l
        self.right = r
```

```
balsa = TreeNode("BALSA", None, None)
aspen = TreeNode("ASPEN", None, balsa)
birch = TreeNode("BIRCH", None, None)
ebony = TreeNode("EBONY", None, None)
maple = TreeNode("MAPLE", ebony, None)
cedar = TreeNode("CEDAR", birch, maple)
beech = TreeNode("BEECH", aspen, cedar)
```

we can!

the special
__init__
method allows us
to both **construct**
a new object and
initialize its state

```
class TreeNode:
    def __init__(self, lbl, l, r):
        self.label = lbl
        self.left = l
        self.right = r
```

```
balsa = TreeNode("BALSA", None, None)
aspen = TreeNode("ASPEN", None, balsa)
birch = TreeNode("BIRCH", None, None)
ebony = TreeNode("EBONY", None, None)
maple = TreeNode("MAPLE", ebony, None)
cedar = TreeNode("CEDAR", birch, maple)
beech = TreeNode("BEECH", aspen, cedar)
```

the `__init__` method
is often referred to as a
constructor

```
class TreeNode:
    def __init__(self, lbl, l, r):
        self.label = lbl
        self.left = l
        self.right = r
```

```
balsa = TreeNode("BALSA", None, None)
aspen = TreeNode("ASPEN", None, balsa)
birch = TreeNode("BIRCH", None, None)
ebony = TreeNode("EBONY", None, None)
maple = TreeNode("MAPLE", ebony, None)
cedar = TreeNode("CEDAR", birch, maple)
beech = TreeNode("BEECH", aspen, cedar)
```


by default, the representation of an object is cold and uninviting

```
class TreeNode:
    def __init__(self, lbl, l, r):
        self.label = lbl
        self.left = l
        self.right = r
```

```
In [2]: balsa = TreeNode("BALSA", None, None)
In [3]: aspen = TreeNode("ASPEN", None, balsa)
In [4]: balsa
Out[4]: <trees.TreeNode at 0x104a17680>
In [5]: aspen
Out[5]: <trees.TreeNode at 0x104a17da0>
```

we can change this by implementing
the `__repr__` method

```
class TreeNode:
```

```
    def __init__(self, lbl, l, r):
```

```
        self.label = lbl
```

```
        self.left = l
```

```
        self.right = r
```

```
    def __repr__(self):
```

```
        return 'TreeNode("' + self.label + '")'
```



```
In [2]: balsa = TreeNode("BALSA", None, None)
```

```
In [3]: aspen = TreeNode("ASPEN", None, balsa)
```

```
In [4]: balsa
```

```
Out[4]: TreeNode("BALSA")
```

```
In [5]: aspen
```

```
Out[5]: TreeNode("ASPEN")
```

```
class TreeNode:
```

```
    def __init__(self, lbl, l, r):
```

```
        self.label = lbl
```

```
        self.left = l
```

```
        self.right = r
```

```
    def __repr__(self):
```

```
        def repr_helper(node, indent):
```

```
            if node == None:
```

```
                return ""
```

```
            result = str(node.label) + "\n"
```

```
            if node.left != None:
```

```
                left_str = repr_helper(node.left, indent+2)
```

```
                result += (" " * indent) + "--L--> " + left_str
```

```
            if node.right != None:
```

```
                right_str = repr_helper(node.right, indent+2)
```

```
                result += (" " * indent) + "--R--> " + right_str
```

```
            return result
```

```
        return repr_helper(self, 0).strip()
```

(fancier `__repr__`
method)

```
In [1]: from trees import *
```

```
In [2]: balsa = TreeNode("BALSA", None, None)
```

```
In [3]: aspen = TreeNode("ASPEN", None, balsa)
```

```
In [4]: balsa
```

```
Out[4]: BALSA
```

```
In [5]: aspen
```

```
Out[5]:
```

```
ASPEN
```

```
--R--> BALSA
```

suppose we wrote a **method** to
count the nodes of a tree

```
class TreeNode:

    def __init__(self, lbl, l, r):
        self.label = lbl
        self.left = l
        self.right = r

    def size(self):
        num_nodes = 1
        if self.left != None:
            num_nodes += self.left.size()
        if self.right != None:
            num_nodes += self.right.size()
        return num_nodes
```

In [2]: beech

Out[2]:

BEECH

--L--> ASPEN

--R--> BALSA

--R--> CEDAR

--L--> BIRCH

--R--> MAPLE

--L--> EBONY

In [3]: beech.size()

Out[3]: 7

the `__len__` method

allows us to use the built-in `len` function

```
class TreeNode:

    def __init__(self, lbl, l, r):
        self.label = lbl
        self.left = l
        self.right = r

    def __len__(self):
        num_nodes = 1
        if self.left != None:
            num_nodes += len(self.left)
        if self.right != None:
            num_nodes += len(self.right)
        return num_nodes
```

```
In [2]: beech
```

```
Out[2]:
```

```
BEECH
```

```
--L--> ASPEN
```

```
      --R--> BALSA
```

```
--R--> CEDAR
```

```
      --L--> BIRCH
```

```
      --R--> MAPLE
```

```
          --L--> EBONY
```

```
In [3]: len(beech)
```

```
Out[3]: 7
```

commonly used special methods

`--repr--`

`--len--`

`--contains--`

`--eq--`

```
In [2]: cedar
Out[2]:
CEDAR
--L--> BIRCH
--R--> MAPLE
--L--> EBONY
```

```
In [3]: len(cedar)
Out[3]: 4
```

```
In [4]: "BIRCH" in cedar
Out[4]: True
```

```
In [5]: "BALSA" in cedar
Out[5]: False
```

```
In [6]: cedar == birch
Out[6]: False
```

```
In [7]: cedar == cedar
Out[7]: True
```